

Tecnatura Superior en Desarrollo de Software

Anexo 05e — El patrón Guard

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

¿Qué es el patrón Guard?

Guard / Guard Clause

Patrón de programación
defensiva

Capa: Application

El patrón **Guard** — también conocido como *Guard Clause* o *cláusula guardiana* — es una técnica de programación defensiva que consiste en verificar condiciones al comienzo de una operación y detener su ejecución inmediatamente si esas condiciones no se cumplen, en lugar de continuar con datos potencialmente inválidos o peligrosos.

El nombre proviene de la idea de un guardia que custodia la entrada de un lugar: antes de que alguien entre, el guardia verifica que cumple los requisitos. Si no los cumple, no entra.

Claves:

- Verifica precondiciones técnicas
- Salida anticipada ante datos inválidos
- Vive en la capa Application
- No contiene reglas de negocio

Dos formas del Guard:

Guard Clause — forma clásica: una condición, una salida anticipada.

```
if (condición inválida) → return / throw
```

Guard Builder — forma moderna: múltiples condiciones, lista de errores acumulados.

```
Guard.Against(valor, campo).Regla1().Regla2() → IEnumerable<ValidationError>
```

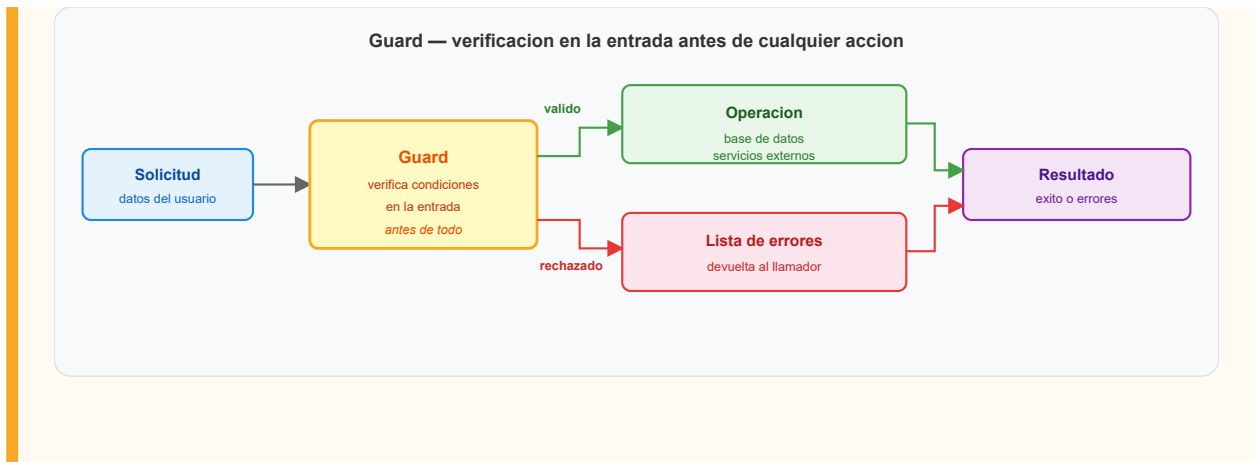
En el curso usamos la forma moderna — el **Guard Builder** — que extiende la idea original para acumular todos los errores en lugar de detenerse en el primero. Pero para entender bien por qué existe y por qué se usa, es fundamental conocer primero la forma clásica.

Analogía del mundo real

Imaginá el ingreso a una sala de cirugía. Antes de que el cirujano ponga un pie en el quirófano, hay una lista de verificaciones obligatorias: lavado de manos, ropa estéril, identificación del paciente, verificación del procedimiento. Si alguna falla — la operación no comienza.

El control no se hace en el medio de la cirugía ni al final. Se hace **en la entrada**, antes de que cualquier acción irreversible ocurra. Esto protege al paciente, al médico y al sistema.

El Guard funciona igual en el software: verifica las condiciones **en la entrada del método o caso de uso**, antes de que cualquier operación con efectos secundarios — consultas a la base de datos, modificaciones de estado, llamadas a servicios externos — comience a ejecutarse.



Características esenciales

La Guard Clause clásica

La forma más simple de Guard es una cláusula condicional al inicio de un método que verifica una precondición. Si no se cumple, el método termina inmediatamente. Esto elimina el anidamiento profundo de `if` que aparece cuando la lógica principal queda dentro de múltiples condiciones.

SIN GUARD CLAUSE

Lógica principal anidada — difícil de leer a medida que crecen las condiciones.

```
1 public void ProcesarPedido(Pedido? pedido)
2 {
3     if (pedido is not null)
4     {
5         if (pedido.Items.Count > 0)
6         {
7             // lógica real - muy anidada
8             Enviar(pedido);
9         }
10    }
11 }
```

CON GUARD CLAUSE

Condiciones inválidas al inicio — lógica principal sin anidamiento.

```
1 public void ProcesarPedido(Pedido? pedido)
2 {
3     // Guard Clause – salida anticipada si la condición falla
4     if (pedido is null) return;
5     if (pedido.Items.Count == 0) return;
6
7     // lógica principal – sin anidamiento
8     Enviar(pedido);
9 }
```

Guard vs. excepción — cuándo usar cada uno

La Guard Clause clásica puede terminar de dos maneras: lanzando una excepción o retornando silenciosamente. La elección depende del contexto y de quién llama al método.

```
1 // Guard que lanza excepción – para métodos internos del dominio
2 // el llamador no debería pasar null nunca – es un error de programación
3 public void Registrar(Libro? libro)
4 {
5     if (libro is null)
6         throw new ArgumentNullException(nameof(libro));
7 }
8
9 // Guard que retorna errores – para casos de uso con input del usuario
10 // el usuario puede equivocarse – no es un error de programación
11 var guard = Guard.Against(titulo, "Titulo").NotNullOrEmpty();
12 if (!guard.IsValid) // acumula errores – no lanza excepción
13     return;
```

Regla de decisión:

Excepción → el dato inválido es un *error de programación* (nunca debería ocurrir si el código está bien escrito).

Lista de errores → el dato inválido es un *error del usuario* (puede ocurrir y debe comunicarse de forma amigable).

Guard como precondition — no como regla de negocio

Una **precondición** es una condición que debe cumplirse antes de que una operación pueda ejecutarse correctamente. Es diferente de una **regla de negocio** — que expresa una restricción del dominio del problema.

Precondicion tecnica vs. Regla de negocio

Precondicion tecnica — Guard

El campo nombre no esta vacio
El campo email tiene algun contenido
El Guid no es el Guid vacio
La fecha no es DateTime.MinValue

capa: Application
resultado: IEnumerable de ValidationError

Regla de negocio — Value Object

El nombre tiene entre 2 y 100 caracteres
El email tiene formato usuario@dominio.com
El cliente es mayor de 18 anos
El DNI tiene 7 u 8 digitos

capa: Domain
resultado: DomainException especifica

¿Por qué se usa?

El patrón Guard existe porque sin él el código de verificación se mezcla con el código de lógica, produciendo métodos largos con múltiples niveles de anidamiento que son difíciles de leer, testear y mantener. El Guard invierte la estructura: las condiciones inválidas se tratan primero y la lógica principal queda limpia y sin ruido.

El Guard protege tres cosas a la vez:

1. **Al dominio:** ningún dato inválido llega a `Book.Create()` ni a los Value Objects.
2. **Al repositorio:** ninguna consulta a la base de datos se ejecuta con datos que ya se sabe que son incorrectos.
3. **Al usuario:** recibe todos los errores juntos en lugar de descubrirlos uno por uno.

Adicionalmente, el Guard hace que el UseCase sea más fácil de leer: las primeras líneas expresan con claridad qué se requiere para que la operación tenga sentido, antes de que comience cualquier acción con efectos secundarios.

¿Cómo encaja en la arquitectura?

El Guard vive en la capa **Application** porque valida las entradas que llegan desde el mundo exterior hacia el sistema. No pertenece al dominio porque no expresa reglas de negocio. No pertenece a Infrastructure porque no accede a ningún recurso. Es el filtro de la capa de aplicación.

El flujo de ejecución del UseCase con Guard sigue siempre el mismo orden:

```
1 public async Task ExecuteAsync(RegisterBookRequest request)
2 {
3     // PASO 1 – Guard: precondiciones técnicas del DTO
4     var g1 = Guard.Against(request.Title, "Title").NotNullOrEmpty().MaxLength(20
5     0);
6     var g2 = Guard.Against(request.Author, "Author").NotNullOrEmpty();
7     var errors = g1.Errors.Concat(g2.Errors).ToList();
8
9     if (errors.Count > 0)
10    {
11        // notifica errores de validación – no toca el dominio
12        await _outputPort.ValidationErrorsAsync(errors);
13        return;
14    }
15
16    // PASO 2 – Dominio: reglas de negocio en los Value Objects
17    Book book = Book.Create(request.Title, request.Author);
18
19    // PASO 3 – Persistencia y notificación de éxito
20    await _repository.AddAsync(book);
21    await _outputPort.HandleSuccessAsync(...);
22 }
```

Orden obligatorio dentro del UseCase:

1. Guard — condiciones técnicas del DTO
2. Reglas de unicidad — consulta al repositorio si es necesario
3. Construcción de la entidad — dominio valida invariantes
4. Persistencia — solo si todo lo anterior paso
5. Notificación — resultado al OutputPort

Marco teórico

Guard Clause y el principio de salida anticipada

La Guard Clause aplica el principio de **salida anticipada** (early return): en lugar de envolver toda la lógica en condiciones positivas anidadas, el método verifica las condiciones negativas al inicio y retorna inmediatamente si alguna falla. La lógica principal queda al mismo nivel de indentación, sin anidamiento.

Este principio está directamente relacionado con la legibilidad: el cerebro humano procesa mejor el código cuando el "camino feliz" (el caso exitoso) está claramente separado de los casos de error.

Guard en el contexto de Fail Fast

El principio **Fail Fast** establece que un sistema debe detectar y reportar los problemas lo antes posible — antes de que causen daño más profundo. El Guard es la implementación de Fail Fast en la capa Application.

Fail Fast en cada capa:

Application — Guard: detecta datos mal formados en el DTO antes de cualquier operación.

Domain — Value Objects: detectan invariantes violadas al construir la entidad — antes de que el estado inválido exista.

Infrastructure — Base de datos: última línea de defensa con constraints de integridad referencial.

Ejemplo completo en Main() — Guard Clause clásica

```
1  // Main() — Guard Clause clásica con salida anticipada
2  static void RegistrarLibro(string? titulo, string? autor)
3  {
4      // Guard Clauses — verificaciones al inicio
5      if (string.IsNullOrEmpty(titulo))
6      {
7          Console.WriteLine("El titulo es obligatorio.");
8          return; // salida anticipada
9      }
10
11     if (string.IsNullOrEmpty(autor))
12     {
13         Console.WriteLine("El autor es obligatorio.");
14         return; // salida anticipada
15     }
16
17     // lógica principal — sin anidamiento, solo si todo pasó
18     Console.WriteLine($"Registrando: {titulo} de {autor}");
19 }
20
21 // llamadas de prueba desde Main()
22 RegistrarLibro(null, "Borges");           // El titulo es obligatorio.
23 RegistrarLibro("Ficciones", null);       // El autor es obligatorio.
24 RegistrarLibro("Ficciones", "Borges");   // Registrando: Ficciones de Borges
```

Errores comunes

Error 1 — confundir Guard con validación de reglas de negocio

El Guard verifica precondiciones técnicas. Las reglas de negocio — mayoría de edad, formato de DNI, límites de crédito — pertenecen al dominio y nunca deben estar en el Guard.

ERROR COMUN

Regla de negocio en el Guard — la edad es una invariante de dominio.

```
1 // MAL: regla de negocio en Application
2 if (request.BirthDate > DateOnly.FromDateTime(DateTime.Today.AddYears(-18)))
3 {
4     Console.WriteLine("Debe ser mayor de edad.");
5     return;
6 }
```

BUENA PRACTICA

Guard verifica que la fecha llegó. El VO BirthDate valida la mayoría de edad.

```
1 // Guard: precondicion - la fecha no es el valor por defecto
2 var g = Guard.Against(request.BirthDate, "BirthDate")
3     .NotDefault();
4 // el VO BirthDate valida la mayoría de edad en el dominio
```

Error 2 — poner el Guard después de operaciones con efectos secundarios

Si el Guard se ejecuta después de una consulta a la base de datos o de una operación que modifica estado, ya es demasiado tarde. El Guard siempre va primero.

ERROR COMUN

Guard después de la consulta — el daño ya pudo ocurrir.

```
1 // MAL: primero consulta, luego valida
2 var existente = await _repo.FindAsync(id);
3 var g = Guard.Against(titulo, "Titulo").NotNullOrEmpty();
4 if (!g.IsValid) return;
```

BUENA PRACTICA

Guard primero — ninguna operación comienza hasta que los datos son válidos.

```
1 // BIEN: Guard primero, luego operaciones
2 var g = Guard.Against(titulo, "Titulo").NotNullOrEmpty();
3 if (!g.IsValid) return;
4 var existente = await _repo.FindAsync(id);
```

Qué NO hacer

No usar Guard en el dominio — los VOs tienen su propio

mecanismo

QUE NO HACER

Guard dentro de un Value Object — mezcla capas y responsabilidades.

```
1 public sealed record BookTitle : ValueObject
2 {
3     public static BookTitle Create(string value)
4     {
5         // Guard en el dominio - incorrecto
6         var g = Guard.Against(value, "title").NotNullOrEmpty();
7         if (!g.IsValid) throw new Exception("invalido");
8         return new BookTitle(value);
9     }
10 }
```

BUENA PRACTICA

El VO usa su propio Fail Fast — lanza DomainException específica.

```
1 public sealed record BookTitle : ValueObject
2 {
3     public static BookTitle Create(string value)
4     {
5         // excepción de dominio específica - sin Guard
6         if (string.IsNullOrWhiteSpace(value))
7             throw new InvalidBookTitleException(value);
8         return new BookTitle(value);
9     }
10 }
```

Resumen — Guard NUNCA debe:

- Contener reglas de negocio (mayoría de edad, formato de DNI, etc.)
- Ejecutarse después de una operación con efectos secundarios
- Vivir en el Domain o en Infrastructure
- Lanzar excepciones de dominio — solo acumula `ValidationError`



Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Tecnicatura Superior en Desarrollo de Software |
Asignatura: Programación II | Tema: Anexo 05e — El patrón Guard | Versión: 2026